

Relatório Técnico: Detecção de Tumores Cerebrais via Aprendizado de Máquina

Disciplina: Inteligência Artificial / Aprendizado de Máquina

Tema: Detecção de Tumores em Imagens de Ressonância Magnética (RM) do Cérebro

Nome: Ludmila Silva

Vídeo de Apresentação: [Assista no YouTube \(https://youtu.be/vePPzw5Lcxo\)](https://youtu.be/vePPzw5Lcxo)

1. Introdução e Contexto

O diagnóstico precoce e preciso de tumores cerebrais é crucial na oncologia médica. O uso de redes neurais artificiais e visão computacional tem se mostrado um aliado poderoso para auxiliar médicos radiologistas na triagem e classificação de imagens de ressonância magnética (RM).

Este relatório documenta os experimentos realizados para classificar RMs do cérebro humano em quatro categorias distintas utilizando o dataset *Brain Tumor MRI Dataset*:

1. **Glioma** (Tumor das células gliais)
2. **Meningioma** (Tumor nas membranas que envolvem o cérebro)
3. **Tumor Hipofisário** (Tumor na glândula pituitária)
4. **Sem Tumor** (RM saudável)

O conjunto de dados possui um total de 7.200 imagens, divididas entre treino (Training) e teste (Testing). Implementamos e avaliamos duas famílias fundamentais de arquiteturas de Aprendizado de Máquina: **Redes Neurais Convolucionais (CNNs)** e **Vision Transformers (ViTs)**, testando 8 variações de parâmetros e técnicas de redução de overfitting (Dropout e L2).

2. Metodologia e Pré-processamento dos Dados

Para todos os experimentos, adotou-se o seguinte pipeline de processamento de dados em Python com **Keras/TensorFlow** e **Scikit-Learn**:

- **Divisão Treino/Validação:** A pasta **Training** foi subdividida em 80% para treinamento (5.712 imagens) e 20% para validação local (1.428 imagens) utilizando uma semente aleatória fixa (`seed=123`).
- **Avaliação Independente:** O conjunto de dados da pasta **Testing** (1.311 imagens) foi mantido estritamente para o cálculo das métricas de teste final, garantindo que o modelo fosse testado em dados não vistos.
- **Normalização de Pixels:** Os valores originais dos pixels das imagens foram normalizados para a escala $[0, 1]$ dividindo por 255 (`layers.Rescaling(1./255)`).
- **Dimensões da Imagem:**
 - Experimentos 1 a 7: Imagens redimensionadas para $150 \times 150 \times 3$.
 - Experimento 8: Imagens redimensionadas para $300 \times 300 \times 3$ (com o grid de patches adaptado para manter a consistência dimensional).

3. Descrição Detalhada e Códigos das Arquiteturas (Experimentos 1 a 8)

Abaixo estão explicadas as variações estruturais, hiperparâmetros e trechos de código exatos que diferenciam cada um dos 8 experimentos realizados:

Grupo A: Redes Neurais Convolucionais (CNN)

Experimento 1 (CNN Base)

- **Variação:** Arquitetura baseline simples com 3 camadas convolucionais (**Conv2D** com ativação ReLU e filtros **[32, 64, 64]**), cada uma seguida de uma camada de pooling máximo (**MaxPooling2D**). Os mapas de características são achatados (**Flatten**) e conectados a uma camada densa de 64 neurônios, seguida da camada de saída Softmax (4 classes). Sem qualquer regularização de pesos ou dropout.
- **Trecho do Código (Definição do Modelo):**

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_HEIGHT,
IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),

    layers.Dense(64, activation='relu'),
    layers.Dense(4, activation='softmax') # 4 classes de saída
])
```

Experimento 2 (CNN com Dropout)

- **Variação:** Mantém a exata estrutura do baseline (Experimento 1), mas incorpora uma camada de **Dropout(0.5)** logo após a camada densa de 64 neurônios. O objetivo é desativar aleatoriamente 50% dos neurônios a cada iteração de treino, forçando a rede a aprender caminhos redundantes e mitigar o overfitting.
- **Trecho do Código (Definição do Modelo):**

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_HEIGHT,
IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
```

```
layers.Flatten(),

layers.Dense(64, activation='relu'),
layers.Dropout(0.5), # Regularização por dropout adicionada aqui
layers.Dense(4, activation='softmax')
])
```

Experimento 3 (CNN com Regularização L2)

- **Variação:** Aplicação de regularização nos pesos (ℓ_2) com fator de penalização 0.01 em todos os kernels convolucionais e na camada densa principal. A regularização L2 penaliza pesos de valores elevados inserindo a norma quadrada do vetor de pesos no cálculo do loss, forçando o modelo a manter coeficientes pequenos e uniformes.
- **Trecho do Código (Definição do Modelo):**

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu',
                  kernel_regularizer=tf.keras.regularizers.l2(0.01),
                  input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu',
                  kernel_regularizer=tf.keras.regularizers.l2(0.01)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu',
                  kernel_regularizer=tf.keras.regularizers.l2(0.01)),
    layers.Flatten(),

    layers.Dense(64, activation='relu',
                 kernel_regularizer=tf.keras.regularizers.l2(0.01)),
    layers.Dense(4, activation='softmax')
])
```

Experimento 4 (CNN com Aumento de Neurônios na Densa)

- **Variação:** Mesma estrutura convolucional da CNN Base, porém a camada densa oculta final foi expandida de 64 para 256 neurônios, aumentando a capacidade de memorização e representação não linear do classificador final sobre os recursos visuais extraídos.
- **Trecho do Código (Definição do Modelo):**

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_HEIGHT,
IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
```

```

layers.MaxPooling2D((2, 2)),

layers.Conv2D(64, (3, 3), activation='relu'),
layers.Flatten(),

layers.Dense(256, activation='relu'), # Modificado: de 64 para 256 neurônios
layers.Dense(4, activation='softmax')
])

```

Grupo B: Vision Transformers (ViT)

Dada a ampla utilização dos Transformers em visão computacional (ViTs) hoje em dia, implementamos uma arquitetura customizada contendo camadas de extração de patches e encoders posicionais.

Experimento 5 (ViT Base)

- **Variação:** Imagens de 150×150 divididas em patches de 25×25 pixels (36 patches por imagem). Os patches são projetados linearmente para uma dimensão de 64 e somados a embeddings posicionais aprendíveis. Passam por 4 blocos de Transformer Encoder (LayerNorm, Multi-Head Attention com 4 cabeças, conexões residuais e sub-bloco MLP de dimensões [128, 64]). Sem dropout ou L2.
- **Trecho do Código (Definição do Classificador ViT):**

```

def create_vit_classifier():
    inputs = layers.Input(shape=(IMG_HEIGHT, IMG_WIDTH, 3))
    patches = Patches(PATCH_SIZE)(inputs)
    encoded_patches = PatchEncoder(NUM_PATCHES, PROJECTION_DIM)(patches)
    x = encoded_patches
    for _ in range(TRANSFORMER_LAYERS):
        x1 = layers.LayerNormalization(epsilon=1e-6)(x)
        attention_output = layers.MultiHeadAttention(
            num_heads=NUM_HEADS, key_dim=PROJECTION_DIM
        )(x1, x1)
        x2 = layers.Add()([attention_output, x])
        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
        x3 = mlp(x3, hidden_units=TRANSFORMER_UNITS)
        x = layers.Add()([x3, x2])
    representation = layers.LayerNormalization(epsilon=1e-6)(x)
    representation = layers.Flatten()(representation)
    logits = layers.Dense(4, activation="softmax")(representation)
    return tf.keras.Model(inputs=inputs, outputs=logits)

```

Experimento 6 (ViT com Dropout)

- **Variação:** Adiciona taxas de dropout estruturadas no Vision Transformer: `Dropout(0.1)` nos embeddings de patches, `dropout=0.1` nas camadas de atenção, `Dropout(0.3)` nas camadas internas

do MLP do bloco Transformer, e `Dropout(0.5)` no cabeçote densificador de saída para avaliar o efeito regulatório em redes de auto-atenção.

- **Trecho do Código (Definição do Classificador ViT):**

```
def create_vit_classifier():
    inputs = layers.Input(shape=(IMG_HEIGHT, IMG_WIDTH, 3))
    patches = Patches(PATCH_SIZE)(inputs)
    # Adicionado dropout_rate nos patches e embeddings
    encoded_patches = PatchEncoder(NUM_PATCHES, PROJECTION_DIM, dropout_rate=0.1)
    (patches)
    x = encoded_patches
    for _ in range(TRANSFORMER_LAYERS):
        x1 = layers.LayerNormalization(epsilon=1e-6)(x)
        attention_output = layers.MultiHeadAttention(
            num_heads=NUM_HEADS, key_dim=PROJECTION_DIM, dropout=0.1 # Dropout na
Atenção
        )(x1, x1)
        x2 = layers.Add()([attention_output, x])
        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
        x3 = mlp(x3, hidden_units=TRANSFORMER_UNITS, dropout_rate=0.3) # Dropout
no MLP
        x = layers.Add()([x3, x2])
    representation = layers.LayerNormalization(epsilon=1e-6)(x)
    representation = layers.Flatten()(representation)
    representation = layers.Dropout(0.5)(representation) # Head Dropout final
    logits = layers.Dense(4, activation="softmax")(representation)
    return tf.keras.Model(inputs=inputs, outputs=logits)
```

Experimento 7 (ViT com Regularização L2)

- **Variação:** Aplicação de penalização de peso L2 (`0.01`) nas projeções de patches, no Multi-Head Attention, na rede MLP interna do bloco de encoders e na camada linear de classificação de saída, mantendo as dimensões de patch e blocos idênticas ao ViT Base.
- **Trecho do Código (Definição do Classificador ViT):**

```
# Modificações no PatchEncoder e MLP para aplicar regularizador L2
class PatchEncoder(layers.Layer):
    def __init__(self, num_patches, projection_dim, l2_reg=0.01, **kwargs):
        super().__init__(**kwargs)
        self.projection = layers.Dense(units=projection_dim,
kernel_regularizer=tf.keras.regularizers.l2(l2_reg))
        ...

def mlp(x, hidden_units, l2_reg=0.01):
    for units in hidden_units:
        x = layers.Dense(units, activation=tf.nn.gelu,
kernel_regularizer=tf.keras.regularizers.l2(l2_reg))(x)
    return x
```

```
# Estrutura do classificador final com L2 na Atenção e na saída
def create_vit_classifier():
    ...
    for _ in range(TRANSFORMER_LAYERS):
        x1 = ...
        attention_output = layers.MultiHeadAttention(
            num_heads=NUM_HEADS, key_dim=PROJECTION_DIM,
            kernel_regularizer=tf.keras.regularizers.l2(0.01) # L2 na MHA
        )(x1, x1)
        ...
    representation = ...
    logits = layers.Dense(4, activation="softmax",
                           kernel_regularizer=tf.keras.regularizers.l2(0.01))
    (representation) # L2 na Saída
    return tf.keras.Model(inputs=inputs, outputs=logits)
```

Experimento 8 (ViT Otimizado)

- **Variação:** Modelo expandido inspirado nos hiperparâmetros de alto desempenho: resolução espacial ampliada para 300×300 pixels; tamanho de patch de 30×30 (100 patches); projeção de dimensão 128 com 8 cabeças de atenção; otimizador **Adamax** com taxa de aprendizado 0.001; data augmentation nativo de brilho; cabeçote de classificação duplo (`Dropout(0.3) -> Dense(128, relu) -> Dropout(0.25) -> Dense(4)`) e regularização L2 mais sutil ($1e-4$).
- **Trecho do Código (Definição do Classificador ViT Otimizado):**

```
# Hiperparâmetros expandidos: IMG_HEIGHT=300, PATCH_SIZE=30, PROJECTION_DIM=128,
NUM_HEADS=8
# Data Augmentation de Brilho acoplado ao fluxo de treino:
data_augmentation = tf.keras.Sequential([
    layers.RandomBrightness(factor=0.2, value_range=(0.0, 1.0), seed=123)
])

# Estrutura de Classificação Otimizada
def create_vit_classifier():
    inputs = layers.Input(shape=(300, 300, 3))
    patches = Patches(30)(inputs)
    encoded_patches = PatchEncoder(100, 128, dropout_rate=0.1, l2_reg=1e-4)
    (patches)
    x = encoded_patches
    for _ in range(4):
        x1 = layers.LayerNormalization(epsilon=1e-6)(x)
        attention_output = layers.MultiHeadAttention(
            num_heads=8, key_dim=128, dropout=0.1,
            kernel_regularizer=tf.keras.regularizers.l2(1e-4)
        )(x1, x1)
        x2 = layers.Add()([attention_output, x])
        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
        x3 = mlp(x3, hidden_units=[256, 128], dropout_rate=0.1, l2_reg=1e-4)
        x = layers.Add()([x3, x2])
```

```
representation = layers.LayerNormalization(epsilon=1e-6)(x)
representation = layers.Flatten()(representation)

# Cabeçote Duplo Otimizado
representation = layers.Dropout(0.3)(representation)
representation = layers.Dense(128, activation="relu",
kernel_regularizer=tf.keras.regularizers.l2(1e-4))(representation)
representation = layers.Dropout(0.25)(representation)

logits = layers.Dense(4, activation="softmax",
kernel_regularizer=tf.keras.regularizers.l2(1e-4))(representation)
return tf.keras.Model(inputs=inputs, outputs=logits)

# Compilado com Adamax
model.compile(optimizer=tf.keras.optimizers.Adamax(learning_rate=0.001), ...)
```

4. Tabela de Resultados Consolidados

Após executar todos os experimentos no conjunto de teste independente (1.311 imagens), as métricas macro foram compiladas na tabela a seguir:

Experimento	Arquitetura	Variação / Parâmetros	Acurácia	Precisão	Recall	F-Score
Exp 1	CNN	Baseline (3 Conv, 64 Neurônios, Sem Regularização)	0.8719	0.8736	0.8719	0.8698
Exp 2	CNN	Dropout (0.5) na camada densa final	0.8650	0.8697	0.8650	0.8630
Exp 3	CNN	Regularização L2 (0.01) nas camadas Conv e Densa	0.8006	0.8044	0.8006	0.7902
Exp 4	CNN	Aumento de neurônios na Densa (256)	0.8781	0.8827	0.8781	0.8753
Exp 5	ViT	Baseline (Patch 25x25, 4 blocos, Sem Regularização)	0.8356	0.8387	0.8356	0.8323
Exp 6	ViT	Dropout (0.1 no Transformer, 0.3 no MLP, 0.5 no Cabeçote)	0.7819	0.7818	0.7819	0.7747
Exp 7	ViT	Regularização L2 (0.01) nas projeções e MLP	0.7706	0.7742	0.7706	0.7600
Exp 8	ViT	Otimizado (300x300, Adamax, Brilho, Densa Dupla, L2 1e-4)	0.7612	0.7565	0.7613	0.7471

5. Análise Comparativa e Conjecturas Acadêmicas

Qual foi a melhor abordagem aplicada?

A melhor abordagem geral foi o **Experimento 4 (CNN com Aumento de Neurônios na Densa)**, alcançando **87.81% de Acurácia** e **87.53% de F-Score** no conjunto de teste. No grupo dos Vision Transformers, a melhor configuração foi o **Experimento 5 (ViT Base)**, com **83.56% de Acurácia**.

Conjectura 1: Eficiência de Dados e Inductive Biases (CNN vs. ViT)

O fato de as CNNs terem superado sistematicamente os Vision Transformers no teste reflete um princípio fundamental do aprendizado profundo: os **vieses indutivos (inductive biases)**.

- As CNNs possuem viés indutivo de **localidade** (pixels adjacentes são correlacionados) e **invariância à translação** (uma característica detectada no topo esquerdo é igual no canto inferior). Isso torna a CNN altamente eficiente ao aprender com poucos dados (o dataset tem 5.712 imagens de treino).
- Os Vision Transformers (ViTs) não possuem esse viés embutido. Eles dependem do mecanismo de auto-atenção global para aprender as relações de distância entre os patches do zero. Por isso, ViTs são tradicionalmente conhecidos por sofrer de subajuste (*underfitting*) quando treinados do zero em datasets pequenos/médios, necessitando de bases muito maiores (como ImageNet) ou pré-treinamento maciço para superar as CNNs.

Conjectura 2: O Efeito da Regularização (Dropout e L2) no Tempo de Treinamento

Em ambos os modelos (CNN e ViT), a introdução de Dropout ou L2 diminuiu as métricas obtidas na 10ª época (comparando Exp 2 e Exp 3 contra Exp 1; e Exp 6 e Exp 7 contra Exp 5). Isso ocorre porque a regularização adiciona ruído ou restringe os valores de parâmetros para prevenir o overfitting no longo prazo.

Consequentemente, o modelo treina de forma mais lenta e exige **muito mais épocas** (e.g., 30 a 50 épocas) para atingir sua convergência e superioridade. Em um limite curto de apenas 10 épocas, as versões regularizadas sofrem de subajuste relativo (*underfit* temporal), não superando os modelos sem regularização.

Conjectura 3: Impacto do Experimento 8 (ViT Otimizado)

O Experimento 8 possui uma capacidade teórica muito maior (4.3 milhões de parâmetros, imagens a \$300\times300\$ e data augmentation de brilho). No entanto, sua acurácia de teste foi a menor do grupo ViT (76.12%). Conjectura-se que a dimensão de entrada maior aumenta drasticamente a complexidade do espaço de busca dos pesos. Para um Vision Transformer com cerca de 4.3M de parâmetros, treinar de forma eficaz em CPU por apenas 10 épocas é insuficiente para convergir. Modelos maiores necessitam de taxas de aprendizado agendadas (warmup + decay) e de períodos de otimização muito mais longos.

6. Códigos Utilizados como Base

Abaixo estão expostos os scripts base em Python/Keras para os experimentos realizados, que servem de modelo estrutural para a reprodução dos experimentos.

Código Base 1: Pipeline CNN (Exemplo Baseado no Experimento 1)


```
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
import numpy as np
import os
import kagglehub

BATCH_SIZE = 32
IMG_HEIGHT = 150
IMG_WIDTH = 150
EPOCHS = 10

# Caminho local conhecido para evitar falhas de conexão com o Kaggle
LOCAL_DATASET_PATH =
r"C:\Users\Usuario\.cache\kagglehub\datasets\masoudnickparvar\brain-tumor-mri-
dataset\versions\2"
if os.path.exists(LOCAL_DATASET_PATH):
    DATASET_PATH = LOCAL_DATASET_PATH
else:
    DATASET_PATH = kagglehub.dataset_download("masoudnickparvar/brain-tumor-mri-
dataset")

train_dataset = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATASET_PATH, 'Training'),
    validation_split=0.2, subset="training", seed=123,
    image_size=(IMG_HEIGHT, IMG_WIDTH), batch_size=BATCH_SIZE
)
val_dataset = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATASET_PATH, 'Training'),
    validation_split=0.2, subset="validation", seed=123,
    image_size=(IMG_HEIGHT, IMG_WIDTH), batch_size=BATCH_SIZE
)
test_dataset = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATASET_PATH, 'Testing'),
    image_size=(IMG_HEIGHT, IMG_WIDTH), batch_size=BATCH_SIZE, shuffle=False
)

normalization_layer = layers.Rescaling(1./255)
train_dataset = train_dataset.map(lambda x, y: (normalization_layer(x), y))
val_dataset = val_dataset.map(lambda x, y: (normalization_layer(x), y))
test_dataset = test_dataset.map(lambda x, y: (normalization_layer(x), y))

model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_HEIGHT,
IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(4, activation='softmax')
```

```

]))

model.compile(optimizer='adam',

loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
              metrics=['accuracy'])

model.fit(train_dataset, validation_data=val_dataset, epochs=EPOCHS)

# Avaliação
y_true = []
y_pred_probs = []
for images, labels in test_dataset:
    y_true.extend(labels.numpy())
    preds = model.predict(images, verbose=0)
    y_pred_probs.extend(preds)

y_pred = np.argmax(y_pred_probs, axis=1)

acuracia = accuracy_score(y_true, y_pred)
precisao = precision_score(y_true, y_pred, average='macro')
recall = recall_score(y_true, y_pred, average='macro')
f_measure = f1_score(y_true, y_pred, average='macro')

print(f"Acurácia: {acuracia:.4f} | Precisão: {precisao:.4f} | Recall: {recall:.4f}
      | F-Score: {f_measure:.4f}")

```

Código Base 2: Pipeline Vision Transformer (Exemplo Baseado no Experimento 5)

```

import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
import numpy as np
import os
import kagglehub

BATCH_SIZE = 32
IMG_HEIGHT = 150
IMG_WIDTH = 150
EPOCHS = 10
PATCH_SIZE = 25
NUM_PATCHES = (IMG_HEIGHT // PATCH_SIZE) * (IMG_WIDTH // PATCH_SIZE)
PROJECTION_DIM = 64
NUM_HEADS = 4
TRANSFORMER_LAYERS = 4
TRANSFORMER_UNITS = [128, 64]

# Caminho local conhecido para evitar falhas de conexão com o Kaggle
LOCAL_DATASET_PATH =
r"C:\Users\Usuario\.cache\kagglehub\datasets\masoudnickparvar\brain-tumor-mri-

```

```

dataset\versions\2"
if os.path.exists(LOCAL_DATASET_PATH):
    DATASET_PATH = LOCAL_DATASET_PATH
else:
    DATASET_PATH = kagglehub.dataset_download("masoudnickparvar/brain-tumor-mri-
dataset")

train_dataset = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATASET_PATH, 'Training'),
    validation_split=0.2, subset="training", seed=123,
    image_size=(IMG_HEIGHT, IMG_WIDTH), batch_size=BATCH_SIZE
)
val_dataset = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATASET_PATH, 'Training'),
    validation_split=0.2, subset="validation", seed=123,
    image_size=(IMG_HEIGHT, IMG_WIDTH), batch_size=BATCH_SIZE
)
test_dataset = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATASET_PATH, 'Testing'),
    image_size=(IMG_HEIGHT, IMG_WIDTH), batch_size=BATCH_SIZE, shuffle=False
)

normalization_layer = layers.Rescaling(1./255)
train_dataset = train_dataset.map(lambda x, y: (normalization_layer(x), y))
val_dataset = val_dataset.map(lambda x, y: (normalization_layer(x), y))
test_dataset = test_dataset.map(lambda x, y: (normalization_layer(x), y))

class Patches(layers.Layer):
    def __init__(self, patch_size, **kwargs):
        super().__init__(**kwargs)
        self.patch_size = patch_size

    def call(self, images):
        batch_size = tf.shape(images)[0]
        patches = tf.image.extract_patches(
            images=images,
            sizes=[1, self.patch_size, self.patch_size, 1],
            strides=[1, self.patch_size, self.patch_size, 1],
            rates=[1, 1, 1, 1],
            padding="VALID",
        )
        patch_dims = patches.shape[-1]
        patches = tf.reshape(patches, [batch_size, -1, patch_dims])
        return patches

class PatchEncoder(layers.Layer):
    def __init__(self, num_patches, projection_dim, **kwargs):
        super().__init__(**kwargs)
        self.num_patches = num_patches
        self.projection = layers.Dense(units=projection_dim)
        self.position_embedding = layers.Embedding(
            input_dim=num_patches, output_dim=projection_dim
        )

```

```
def call(self, patch):
    positions = tf.range(start=0, limit=self.num_patches, delta=1)
    return self.projection(patch) + self.position_embedding(positions)

def mlp(x, hidden_units):
    for units in hidden_units:
        x = layers.Dense(units, activation=tf.nn.gelu)(x)
    return x

def create_vit_classifier():
    inputs = layers.Input(shape=(IMG_HEIGHT, IMG_WIDTH, 3))
    patches = Patches(PATCH_SIZE)(inputs)
    encoded_patches = PatchEncoder(NUM_PATCHES, PROJECTION_DIM)(patches)
    x = encoded_patches
    for _ in range(TRANSFORMER_LAYERS):
        x1 = layers.LayerNormalization(epsilon=1e-6)(x)
        attention_output = layers.MultiHeadAttention(
            num_heads=NUM_HEADS, key_dim=PROJECTION_DIM
        )(x1, x1)
        x2 = layers.Add()([attention_output, x])
        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
        x3 = mlp(x3, hidden_units=TRANSFORMER_UNITS)
        x = layers.Add()([x3, x2])
    representation = layers.LayerNormalization(epsilon=1e-6)(x)
    representation = layers.Flatten()(representation)
    logits = layers.Dense(4, activation="softmax")(representation)
    return tf.keras.Model(inputs=inputs, outputs=logits)

model = create_vit_classifier()
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),

loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
              metrics=['accuracy'])

model.fit(train_dataset, validation_data=val_dataset, epochs=EPOCHS)

# Avaliação finalizada omitida por brevidade (idêntica ao pipeline da CNN)
```